

Caso semestral

Sigla	Nombre Asignatura
DSY1106	DESARROLLO FULLSTACK III

Caso Semestral: "SmartLogix – Plataforma Inteligente para la Gestión Logística de eCommerce"

El auge del comercio electrónico ha impulsado la necesidad de sistemas más eficientes y escalables para la gestión de inventarios, pedidos y envíos. SmartLogix es una startup en crecimiento que busca ofrecer una plataforma moderna e integrada para la optimización de procesos logísticos en pequeñas y medianas empresas (PYMES) de eCommerce. La empresa ha identificado que muchas de estas PYMEs aún dependen de sistemas monolíticos y procesos manuales, lo que genera inconsistencias en la información, retrasos en la entrega y dificultades para manejar picos de demanda.

Para resolver estos problemas, SmartLogix quiere desarrollar una solución basada en microservicios, con una capa frontend moderna y flexible, y un backend escalable y seguro, permitiendo a las PYMEs gestionar su operación logística con mayor eficiencia y reducir costos operativos.

El desarrollo de este sistema se llevará a cabo en **tres etapas**, alineadas con las evaluaciones parciales del curso. Finalmente, en el **Examen Final Transversal**, los/as estudiantes consolidarán su solución integrando todos los módulos desarrollados en un sistema funcional.

Sección 1: Diseño de Arquitectura y Patrones de Microservicios (Parcial 1)

Contexto

Actualmente, las empresas que utilizan soluciones logísticas tradicionales enfrentan desafíos como **sincronización deficiente de inventarios, errores en la asignación de pedidos, y demoras en la coordinación de envíos**. Muchos de estos problemas surgen porque sus sistemas monolíticos no permiten automatización ni integración con nuevas tecnologías. SmartLogix ha decidido desarrollar una **arquitectura moderna basada en microservicios** para garantizar la escalabilidad y flexibilidad de su plataforma, facilitando la automatización de la gestión logística.

La solución debe contemplar tres módulos principales:

- **Gestión de Inventario:** Mantener actualizados los niveles de stock en tiempo real, optimizando la sincronización entre múltiples bodegas y tiendas.

- **Procesamiento de Pedidos:** Automatizar la validación, aprobación y asignación de pedidos, asegurando trazabilidad y reduciendo errores.
- **Coordinación de Envíos:** Mejorar la planificación de rutas y la comunicación con transportistas, garantizando tiempos de entrega eficientes.

Requerimientos Técnicos

Los/as estudiantes deberán diseñar una **arquitectura de microservicios escalable**, aplicando **patrones de diseño y arquetipos arquitectónicos** que permitan la modularización del sistema. Para ello, deberán:

- **Definir los microservicios clave**, asegurando separación de responsabilidades y escalabilidad.
- Diseñar una **API Gateway** que gestione la comunicación entre microservicios y el frontend.
- Implementar patrones como **Repository Pattern** para la persistencia de datos, **Factory Method** para la creación de instancias y **Circuit Breaker** para manejar fallos en la comunicación entre servicios.
- Asegurar que los servicios sean **escalables y desacoplados**, permitiendo futuras mejoras sin afectar el funcionamiento del sistema.
- Documentar las decisiones arquitectónicas y justificar la selección de patrones.

Al finalizar esta etapa, los equipos deberán presentar un informe con la propuesta de arquitectura, un diagrama detallado de los microservicios y una justificación de los patrones seleccionados.

Sección 2: Desarrollo de Componentes Frontend y Backend (Parcial 2)

Contexto

Después de definir la arquitectura del sistema, SmartLogix busca desarrollar una primera versión funcional que permita validar la propuesta tecnológica. La empresa requiere un sistema que cuente con una **interfaz de usuario intuitiva y responsiva**, permitiendo que los administradores de las PYMEs gestionen su operación logística de manera sencilla. Al mismo tiempo, el backend debe ser capaz de manejar múltiples solicitudes concurrentes sin comprometer el rendimiento.

Además, para mejorar la experiencia de los/as usuarios/as, SmartLogix quiere que la plataforma pueda **integrarse con marketplaces y transportistas**, facilitando la automatización del procesamiento de pedidos y envíos.

Requerimientos Técnicos

Los/as estudiantes deberán desarrollar la solución con los siguientes elementos clave:

- **Frontend:** Implementar una interfaz construida con un framework moderno (React, Angular o Vue.js), asegurando que la comunicación con el backend se realice vía API REST. Los componentes frontend deberán ser empaquetados como **módulos NPM** reutilizables.
- **Backend:** Construir al menos tres componentes backend utilizando **arquetipos Maven personalizados**:
 - Un **Backend For Frontend (BFF)** para gestionar la interacción entre el frontend y los microservicios.
 - Dos **microservicios independientes** para gestionar Inventario y Pedidos, conectados a bases de datos mediante JPA.
- **Conexión con Bases de Datos:** Utilizar **JPA y entidades**, asegurando la persistencia de datos. También se podrán utilizar procedimientos almacenados (SPs) para optimizar operaciones.
- **Versionamiento del Código:** Todos los componentes deberán ser versionados en **Git**, utilizando estrategias de branching como Git Flow o GitHub Flow para facilitar el trabajo colaborativo.
- **Implementación de Patrones de Diseño:** Aplicar patrones adecuados para mejorar la organización y mantenibilidad del código, asegurando que los servicios sean modulares y fácilmente extensibles.

Como resultado de esta etapa, los/as estudiantes deberán entregar la primera versión funcional del sistema, acompañada de una presentación donde expliquen las decisiones técnicas, la implementación de los componentes y la integración entre frontend y backend.

Sección 3: Integración, Pruebas Unitarias y Presentación Final (Parcial 3)

Contexto

Con el sistema en funcionamiento, SmartLogix quiere asegurar que la solución sea robusta y confiable antes de su implementación definitiva. Para ello, se requiere que el código desarrollado cumpla con **estándares de calidad**, aplicando **pruebas unitarias** y validando la **cobertura del código** antes del lanzamiento.

En esta última fase, la empresa evaluará cómo se integran los diferentes módulos del sistema, garantizando que la plataforma sea escalable y esté lista para su despliegue en producción.

Requerimientos Técnicos

En esta etapa, los/as estudiantes deberán:

- **Realizar Pruebas Unitarias:** Implementar pruebas para cada uno de los componentes del sistema, asegurando una cobertura mínima del **60% del código**. La validación de cobertura se realizará utilizando herramientas como **SonarQube**.
- **Refactorización y Mejora del Código:** Revisar el código desarrollado, identificar oportunidades de optimización y aplicar mejoras siguiendo buenas prácticas de desarrollo.
- **Despliegue y Ejecución de Pruebas:** Integrar las pruebas unitarias en un pipeline de **Integración Continua**, asegurando que se ejecuten automáticamente en cada actualización del código.
- **Documentación Final:** Completar la documentación del sistema, incluyendo diagramas actualizados, descripción de la arquitectura final y detalles sobre la implementación de pruebas.

Presentación Final

Cada equipo presentará su solución ante el/la docente, abordando los siguientes puntos clave:

- **Explicación de la arquitectura del sistema y decisiones técnicas.**
- **Demostración del funcionamiento del software, mostrando la integración entre frontend y backend.**
- **Estrategia de pruebas implementada y validación de cobertura.**
- **Reflexión sobre los desafíos enfrentados y aprendizajes adquiridos.**

Esta presentación servirá como cierre del curso, permitiendo evaluar de manera integral la capacidad de los/as estudiantes para diseñar, desarrollar, integrar y validar una solución de software basada en **microservicios, patrones de diseño y pruebas automatizadas**.

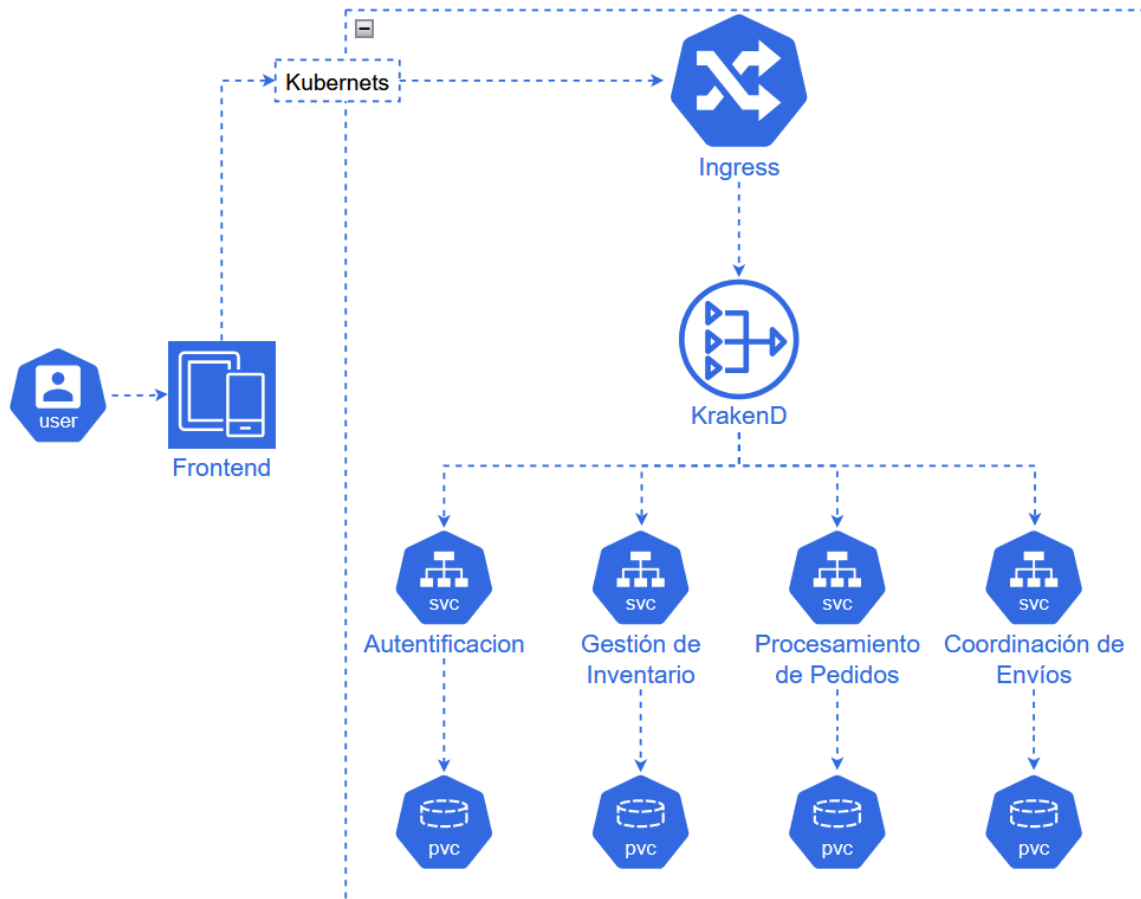
Desarrollo Parcial 1

Utilizaremos Docker, dockerfile, postman, java, springboot, kubernetes(corends), ingress, traefik, cluster, vpc, deployment y base de datos para cada microservicio, API Gateway con Krakend(circuit breaker). Interfaz web con Spring Boot.

Microservicios para agregar:

- **Autenticación (BD):** Gestionar el inicio de sesión, registro de usuarios.
- **Gestión de Inventario (BD):** Mantener actualizados los niveles de stock en tiempo real, optimizando la sincronización entre múltiples bodegas y tiendas.

- **Procesamiento de Pedidos (BD):** Automatizar la validación, aprobación y asignación de pedidos, asegurando trazabilidad y reduciendo errores.
- **Coordinación de Envíos (BD):** Mejorar la planificación de rutas y la comunicación con transportistas, garantizando tiempos de entrega eficientes.



1.- Análisis de la arquitectura actual.

Seguridad: La seguridad principalmente se está abordando mediante el microservicio de autenticación y el uso de un api gateway(Kraked) que da el acceso a los servicios.

Privacidad: La privacidad es importante y es lo que más se debe resguardar debido a los datos sensibles como los datos de autenticación y pedidos.

Sostenibilidad: En sostenibilidad no existen problemas ya que al trabajar con docker, kubernetes, microservicios, uso de cluster y vpc, todo esto da una alta optimización al sistema.

2.- Identificación de riesgos éticos.

2.1 Posibles vulnerabilidades de seguridad en la exposición de servicios

¿Qué significa ?

Al dividir un sistema monolítico en varios microservicios y conectarlos con un frontend, se crean múltiples endpoints (puertas de entrada) y rutas de comunicación, si estos servicios quedan expuestos, el sistema se vuelve vulnerable a ataques.

medidas cautelosas a tener en cuenta :

analizar el bff (backend for frontend) si están configurados para centralizar las peticiones externas, evitando que alguien pueda acceder directamente a los microservicios internos como inventario o pedidos, desde internet

(se ocupará un api gateway)

Notas:

Agregar ejemplo de Historias de Usuario -listo-

¿Cuánto almacenamiento se puede duplicar?

Patrones de Diseños, mostrar casos de uso -listo-

SCL (se ve en HTTPS)

responder preguntas de documentos en el informe no de forma explícita, referenciar a 1.3 y 1.4

principales beneficios del BFF :

optimización de datos : este compila información de múltiples servicios, enviando al cliente solo lo necesario, lo que mejora el rendimiento

independencia : los equipos frontend y backend trabajan de manera más eficiente, reduciendo el acoplamiento

adaptabilidad . ajusta la lógica y el formato de datos dependiendo del dispositivo ya sea móvil o escritorio

seguridad : gestiona tokens y sesiones centralizadas, básicamente En un esquema de microservicios tradicional, el cliente (tu app) suele lidiar con tokens de acceso (JWT) y

debe enviarlos a cada servicio. Con un BFF, la seguridad se vuelve mucho más robusta y sencilla para el frontend mediante un patrón llamado Token Handler.

(se ocupará un api gateway)

¿Qué es un **JWT**?

<https://medium.com/@tericcabrel/implement-jwt-authentication-in-a-spring-boot-3-applications-5839e4fd8fac>

Es un pasaporte digital que el usuario presenta cada vez que quiere entrar a una "frontera" (un microservicio o el backend).

la solución implementada para este riesgo planteado es el BFF como token handler , con este patrón le quitamos la responsabilidad de manejar tokens al frontend y se la entregamos al bff.

en el proyecto actual funcionaria de esta manera

1.- login : el usuario ingresa sus credenciales en el frontend . El frontend envía estos datos únicamente al bff , no al frontend.

2.- generación de token : El BFF toma esas credenciales y se comunica internamente con tu micro servicio de Autenticación(BD)
(pasando a través de tu API Gateway con Krakend. El microservicio valida los datos y genera el token JWT, devolviéndole al BFF.

3.- Sesión Segura (Protección del Frontend) : Una vez que el BFF recibe el JWT, no se lo envía al frontend. En su lugar, el BFF guarda el JWT internamente y le entrega al frontend una cookie de sesión segura (HttpOnly). De esta forma, el token real nunca queda expuesto en el navegador, protegiendo al sistema contra robos de credenciales por ataques cibernéticos

4.-Peticiones de los usuarios: Cuando el frontend necesita interactuar con otros módulos (por ejemplo, consultar el stock en Gestión de Inventario (BD) o crear un envío en Coordinación de Envíos (BD), envía la solicitud al BFF usando su cookie segura. El BFF la recibe, le "inyecta" el verdadero token JWT que tiene guardado y la encamina a través de Krakend hacia el microservicio correspondiente.

de todas formas se ocupará un api gateway en la autenticación del usuario correspondiente

2.2 Manejo de datos personales (Privacidad)

Database per Service : cada microservicio (autenticación, inventario, pedidos, envíos) tiene su propia base de datos , esta medida ética es clave , ya que si se llegara a vulnerar la base de datos de “inventario” no podrá acceder a los datos personas que se encuentran alojados dentro de la base de datos de autenticación

2.3 Riesgo Ético: Impacto ambiental del despliegue y uso de infraestructura (Sostenibilidad)

El riesgo actual mitiga en toda la arquitectura en la nube , ya que esta consume electricidad real en servidores físicos, si el código es ineficiente o si la infraestructura está mal configurada , los servidores trabajan al 100% de su capacidad las 24 horas del día , esto genera un consumo excesivo de energía , lo que va en contra de la sostenibilidad tecnológica.

¿Cómo mitigarlo?

Autoescalado eficiente con Kubernetes y Docker : Éticamente esta es una gran medida , ya que en eventos de alta demanda logística como un cyberday kubernetes detectara el pico de tráfico y levantará más contenedores docker únicamente para el microservicio de “ procesamiento de pedidos “ dejando los demás intactos , cuando la demanda baje , kubernetes destruirá estos contenedores , liberando recursos computacionales y ahorrando energía eléctrica.

3.- Ajustes y mejoras en el diseño.

1. Seguridad: Se debe aplicar un RBAC(Control de roles) esto es para definir qué puede hacer cada usuario. Agregar un HTTPS para encriptar la información. Sistema moderno de token como JWT. Agregar Rate limiting esto para controlar el número de peticiones que se puede realizar.
2. Privacidad: Minimizar datos, encriptar datos como también contraseñas, acceso restringido.
3. Sostenibilidad: Optimización de imágenes de docker(algo no tan pesado), prometheus, grafana para controlar

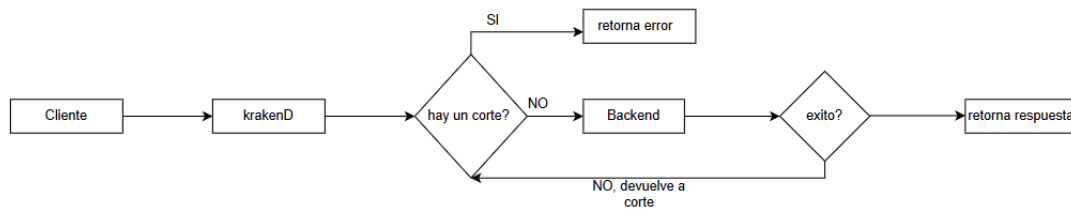
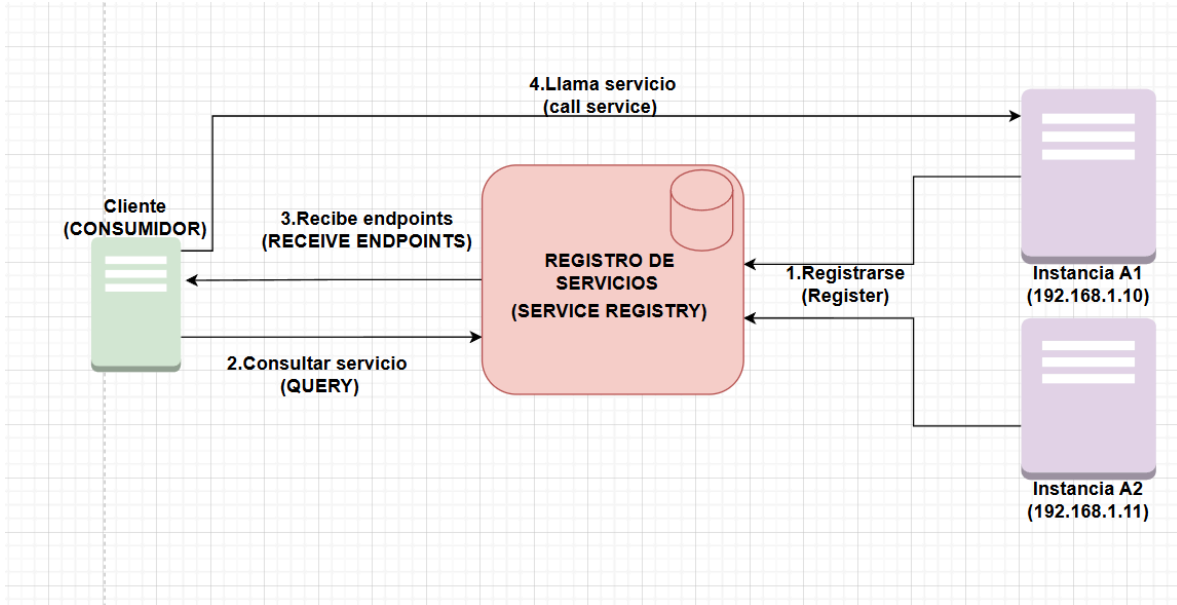


Diagrama circuit breaker

garantizar tiempos de entrega: google maps

comunicacion con transportistas: servicio de mensajeria/sendgrid.

marketplaces.



Descubrimiento de Servicios (Lado del Cliente)

El diagrama representa cómo un cliente encuentra y se comunica con un servicio en una arquitectura distribuida sin conocer previamente su dirección IP, delegando la responsabilidad de la búsqueda al propio cliente.

Registrarse (Register): Las instancias de los servicios (Instancia A1 e Instancia A2) se comunican con el Registro de Servicios en el momento de su arranque. En este paso, notifican sus direcciones de red específicas (192.168.1.10 y 192.168.1.11) para que queden almacenadas en el directorio central.

Consultar servicio (Query): El Cliente (Consumidor) necesita interactuar con el servicio, pero no tiene las direcciones IP de las instancias codificadas en su sistema. Para resolver esto, envía una consulta al Registro de Servicios preguntando por las ubicaciones disponibles para ese servicio en particular.

Recibe endpoints (Receive Endpoints): El Registro de Servicios revisa su directorio interno y le responde al Cliente entregándole una lista con las direcciones IP y puertos (endpoints) de todas las instancias del servicio que se encuentran registradas y operativas.

Llama servicio (Call Service): El Cliente procesa la lista de direcciones recibida, aplica su propia lógica (como un balanceo de carga interno) para seleccionar una de ellas y envía la solicitud final de manera directa a la instancia elegida (en el diagrama, la Instancia A1) estableciendo la comunicación efectiva.

Historias de usuario.

Autenticación.

1. Registro de usuario: Como administrador quiero registrar usuarios en el sistema para dar acceso a operadores y supervisores.

Criterios de aceptación: El administrador puede registrar un usuario con nombre, contraseña, correo y rol. El sistema valida que el correo no esté registrado previamente. La contraseña debe almacenarse encriptada.

2. Inicio de sesión: Como usuario quiero iniciar sesión para entrar a la plataforma según mi rol.

Criterios de aceptación: El usuario debe iniciar sesión con correo y contraseña. El sistema valida credenciales contra la base de datos. Si las credenciales son correctas se genera un token JWT.

3. Gestión de roles: Como administrador quiero registrar usuarios en el sistema para dar acceso a operadores logísticos y supervisores.

Criterios de aceptación: El administrador puede asignar o modificar roles de usuarios. El sistema guarda el rol en la base de datos. Los roles definen permisos sobre inventario, pedidos y envíos.

Inventario.

1. Crear productos: Como administrador quiero crear productos para gestionarlos en el inventario.

Criterio de aceptación: El administrador puede registrar un producto. El producto queda almacenado en la base de datos del microservicio. El sistema confirma la creación exitosa del producto.

2. Consultar stock: Como operador o supervisor quiero consultar el stock disponible.

Criterio de aceptación: El operador consulta el stock disponible. El sistema muestra stock total y stock por bodega. El sistema permite filtrar por producto.

3. Ajustar stock: Como supervisor quiero realizar ajustes manuales del stock para corregir diferencias con el inventario físico.

Criterio de aceptación: El supervisor puede ajustar stock por diferencias físicas. El sistema exige indicar motivo de ajuste. El sistema registra fecha/hora del usuario responsable.

Pedidos.

1. Crear pedido manualmente: Como operador quiero registrar un pedido manualmente para gestionar ventas por redes sociales.

Criterios de aceptación: El operador registra el pedido con el cliente, dirección, productos y cantidades. El sistema valida que los productos existan. El pedido queda almacenado en la base de datos del microservicio de pedidos.

2. Consultar el estado del pedido: Como operador o supervisor quiero consultar el estado del pedido para dar seguimiento y trazabilidad.

Criterios de aceptación: El operador o supervisor puede consultar pedidos por nombre o en listado general. El sistema muestra el estado actual del pedido. El sistema muestra historial de cambios.

Coordinación de envíos.

1. Generar envío: Como operador quiero generar un envío para un pedido aprobado para iniciar el despacho.

Criterios de aceptación: El operador puede generar un envío únicamente para pedidos aprobados. El sistema crea un registro de envío con ID único. El envío inicia en estado "Preparando".

2. Asignar transportista: Como supervisor quiero asignar un transportista al envío para garantizar entrega eficiente.

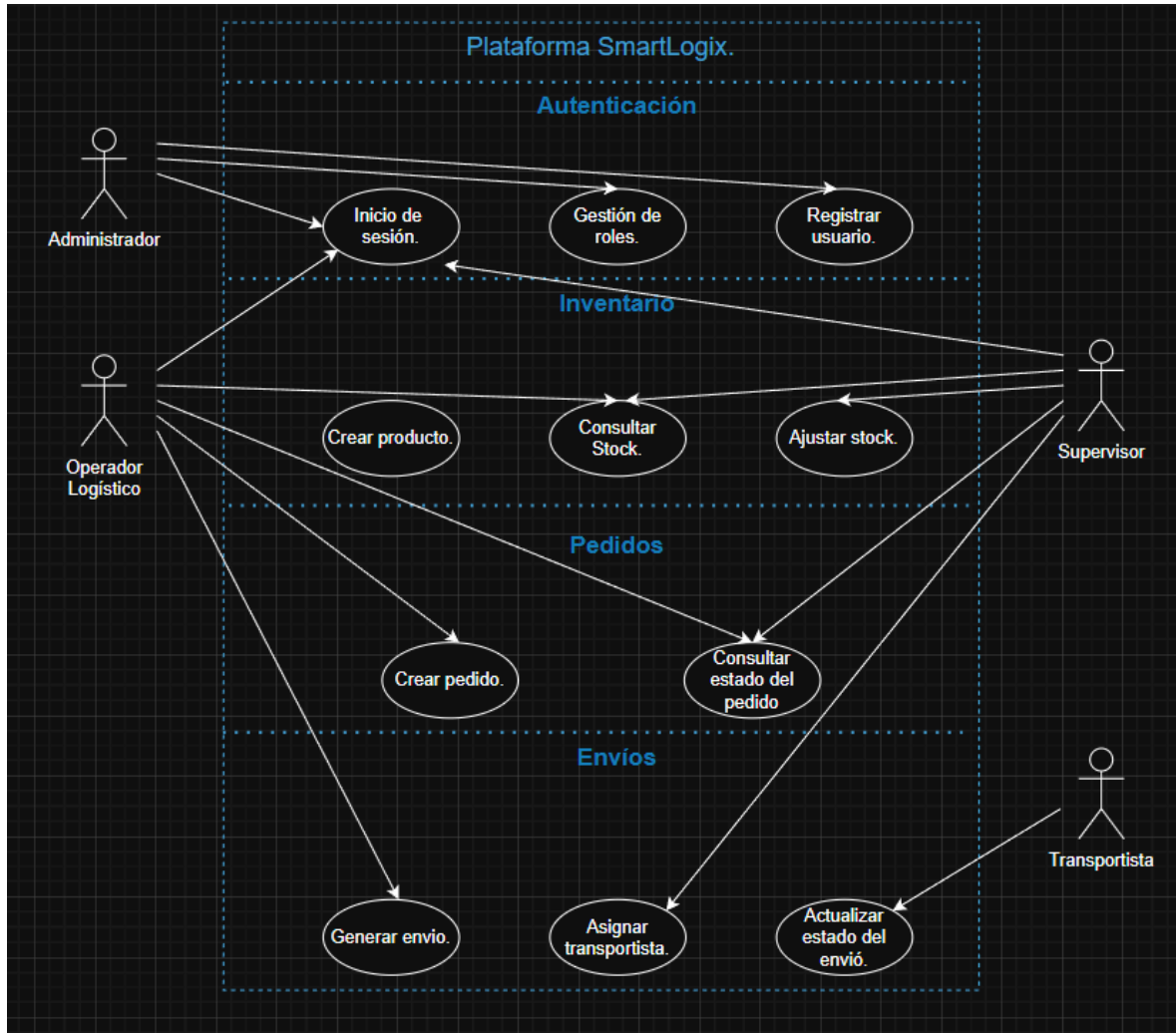
Criterios de aceptación: El supervisor puede asignar un transportista por envío. El sistema guarda el transportista asignado y fecha/hora. El transportista debe poder ver los envíos asignados.

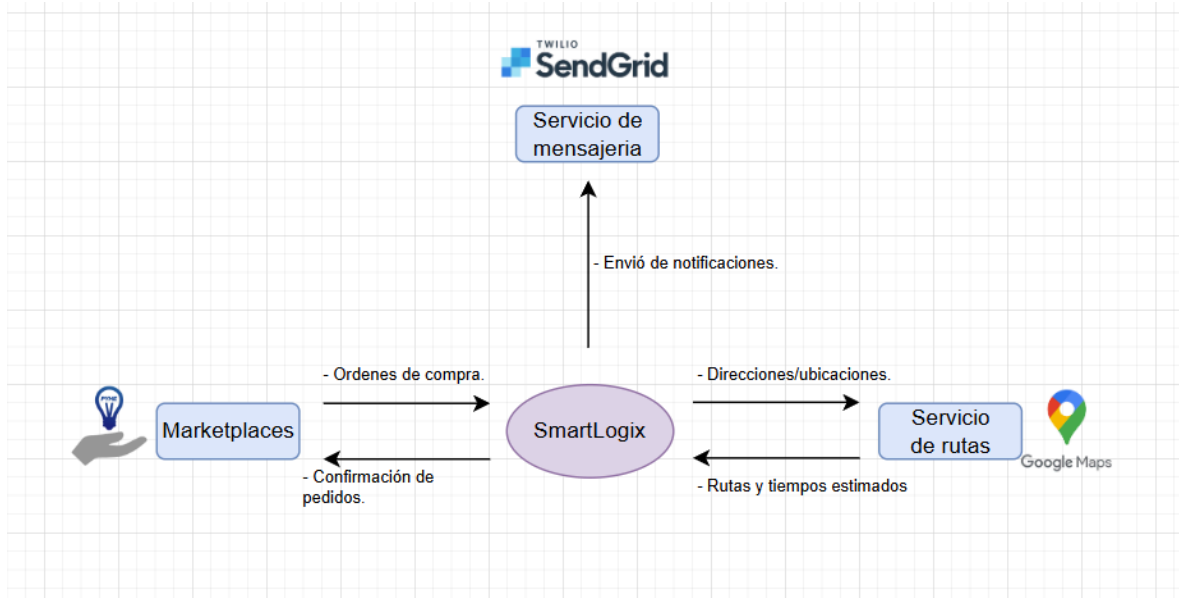
3. Actualizar estado del envío: Como transportista quiero actualizar el estado del envío para mantener trazabilidad.

Criterios de aceptación: El transportista puede actualizar estados: "En ruta", "Retrasado", "Entregado". El sistema guarda el estado actualizado con fecha/hora. Al marcar "Entregado", el pedido asociado cambia de estado a "Entregado".

Caso de uso:

MIERCOLES HASTA LAS 7 PARA ENTREGAR





1.- Análisis de la arquitectura actual

Cada equipo revisará su diseño previo, identificando puntos críticos relacionados con:

- o Seguridad: Métodos de autenticación y control de acceso implementados.
- o Privacidad: Tratamiento de datos sensibles y cumplimiento normativo.
- o Sostenibilidad: Consumo de recursos y optimización de procesos.

2.- Identificación de riesgos éticos

Mediante una lista de verificación y preguntas orientadoras, los/as estudiantes evaluarán su arquitectura en función de:

- o Posibles vulnerabilidades de seguridad en la exposición de servicios.
- o Manejo de datos personales y su alineación con regulaciones como GDPR o HIPAA.
- o Impacto ambiental del despliegue y uso de infraestructura.

3.- Ajustes y mejoras en el diseño

Con base en la evaluación realizada, los equipos deberán:

- o Proponer mejoras en su arquitectura para mitigar los riesgos identificados.
- o Rediseñar diagramas clave para reflejar las modificaciones.
- o Justificar sus decisiones bajo criterios éticos y técnicos.

1. Analisis de arquitectura actual.

2. Identificación de riesgos éticos.

3. Ajustes y mejoras en el diseño.

Paso 1: Revisión y Análisis de Requerimientos

- El/la docente proporcionará un caso práctico que incluye requerimientos funcionales y no funcionales.
- Los equipos analizarán los requerimientos y evaluarán la arquitectura propuesta, identificando:
 - Elementos del diseño que cumplen o no con las especificaciones funcionales.
 - Aspectos éticos como seguridad, privacidad y sostenibilidad.
- **Producto esperado:** Un esquema inicial (puede ser en pizarra o papel) con observaciones sobre el diseño actual.

Paso 2: Identificación de Patrones de Arquitectura

- Los/as estudiantes identificarán patrones de arquitectura que optimicen el diseño actual, justificando por qué son los más adecuados para los requerimientos del cliente.
 - Ejemplo de patrones: **Database-per-Service, Event-Driven Architecture, API Gateway.**
- **Producto esperado:** Una lista priorizada de patrones con justificaciones breves.

Paso 3: Creación de Evidencias y Visualizaciones

- Los equipos generarán evidencias para respaldar su análisis, como:
 - **Diagramas** de arquitectura (pueden ser realizados a mano).
 - Fragmentos de código que demuestren cambios propuestos en endpoints clave (por ejemplo, /inventario y /pedidos).
 - Métricas relevantes para evaluar la escalabilidad o rendimiento del diseño actual.
- **Producto esperado:** Diagrama inicial y un fragmento de código que demuestre al menos un ajuste funcional.

Paso 4: Evaluación Ética y Recomendaciones

- Analizarán cómo el diseño cumple con principios éticos, respondiendo preguntas como:
 - ¿Cómo garantiza la privacidad de los datos de los usuarios?
 - ¿El sistema es sostenible desde el punto de vista computacional?
- Propondrán mejoras concretas al diseño inicial.
- **Producto esperado:** Un listado con al menos tres recomendaciones éticas y técnicas.